

Faculdade Cotemig

Fundamentos Teóricos da Computação

**Trabalho Final — Implementação de Máquinas Abstratas: AFD,
Autômato de Pilha e Máquina de Turing**

Helena Santos Rezende
João Gabriel Rocha Rosa
Gustavo Guimarães Aguiar Lima

Professor: Júlio César da Silva

Belo Horizonte, MG

2026

1. Introdução

A Teoria da Computação é o ramo da ciência da computação que estuda os fundamentos matemáticos do que pode e do que não pode ser computado. No centro dessa teoria estão os modelos formais de computação — abstrações matemáticas que definem o que significa processar informação — entre os quais se destacam o Autômato Finito Determinístico (AFD), o Autômato de Pilha (AP) e a Máquina de Turing (MT).

Cada um desses modelos representa um nível diferente de poder computacional e reconhece uma classe distinta de linguagens formais. O AFD, o mais simples dos três, reconhece as linguagens regulares e opera sem memória auxiliar além de seus próprios estados. O Autômato de Pilha acrescenta uma memória de pilha (estrutura LIFO), o que lhe permite reconhecer as linguagens livres de contexto — uma classe estritamente mais ampla. A Máquina de Turing, por sua vez, é o modelo mais poderoso: com uma fita de memória ilimitada, ela captura tudo aquilo que pode ser computado por qualquer algoritmo, conforme estabelecido pela Tese de Church-Turing.

Este trabalho tem como objetivo implementar simuladores funcionais dessas três máquinas abstratas na linguagem C# (.NET 8), traduzindo fielmente as definições matemáticas formais em código. Para cada máquina foi escolhida uma linguagem-alvo que ilustra seu poder de reconhecimento: $L_1 = \{ w \in \{a,b\}^* \mid w \text{ termina com 'ab'} \}$ para o AFD; $L_2 = \{ a^n b^n \mid n \geq 1 \}$ para o Autômato de Pilha; e $L_3 = \{ a^n b^n c^n \mid n \geq 1 \}$ para a Máquina de Turing.

Além dos simuladores principais, cada parte inclui um desafio complementar: um AFD configurável via arquivo JSON, um segundo Autômato de Pilha para palíndromos, e uma Máquina de Turing que computa a função $f(n) = n + 1$ em representação unária. O trabalho é acompanhado por este relatório técnico, que documenta as decisões de projeto, apresenta os resultados dos testes e discute os limites de cada modelo computacional.

2. Fundamentação Teórica

2.1 Autômato Finito Determinístico (AFD)

Um Autômato Finito Determinístico (AFD) é definido formalmente pela 5-tupla $M = (Q, \Sigma, \delta, q_0, F)$, onde Q é um conjunto finito de estados, Σ é o alfabeto de entrada, $\delta : Q \times \Sigma \rightarrow Q$ é a função de transição total, $q_0 \in Q$ é o estado inicial e $F \subseteq Q$ é o conjunto de estados de aceitação. Uma cadeia w é aceita pelo AFD se a computação a partir de q_0 termina em um estado pertencente a F .

Para a linguagem $L_1 = \{ w \in \{a, b\}^* \mid w \text{ termina com 'ab'} \}$, a 5-tupla é: $Q = \{ q_0, q_1, q_2 \}$, $\Sigma = \{ a, b \}$, $q_0 = q_0$, $F = \{ q_2 \}$, e δ conforme a tabela de transições apresentada na seção 4.1. A expressão regular equivalente a L_1 é $(a \mid b)^* ab$.

Cálculo manual — cadeia 'ab' (esperado: ACEITA):

1. Estado inicial: q_0 . Lê 'a' $\rightarrow \delta(q_0, a) = q_1$.
2. Estado atual: q_1 . Lê 'b' $\rightarrow \delta(q_1, b) = q_2$.
3. Cadeia consumida. Estado final: $q_2 \in F \rightarrow$ **ACEITA**.

Cálculo manual — cadeia 'ba' (esperado: REJEITA):

1. Estado inicial: q_0 . Lê 'b' $\rightarrow \delta(q_0, b) = q_0$.
2. Estado atual: q_0 . Lê 'a' $\rightarrow \delta(q_0, a) = q_1$.
3. Cadeia consumida. Estado final: $q_1 \notin F \rightarrow$ **REJEITA**.

2.2 Autômato de Pilha (AP)

Autômato de Pilha — Definição Formal e Aceitação por Pilha Vazia

Definição formal: a 7-tupla

Um autômato de pilha é definido como uma 7-tupla $AP = (Q, \Sigma, \Gamma, \delta, q_0, Z_0, F)$, onde:

- Q — conjunto finito de estados do autômato.
- Σ — alfabeto de entrada; símbolos que o AP lê da fita.
- Γ — alfabeto da pilha; símbolos que podem ser empilhados ($\Sigma \subseteq \Gamma$ é permitido).
- δ — função de transição: $\delta : Q \times (\Sigma \cup \{\epsilon\}) \times \Gamma \rightarrow \wp(Q \times \Gamma^*)$. Recebe estado

atual, símbolo lido (ou ϵ) e topo da pilha; retorna um conjunto de pares (novo estado, string a empilhar).

- q_0 — estado inicial, $q_0 \in Q$.
 - Z_0 — símbolo inicial da pilha (marcador de fundo), $Z_0 \in \Gamma$.
 - F — conjunto de estados de aceitação, $F \subseteq Q$.
-

Aceitação por pilha vazia

A palavra w é aceita se existe uma sequência de transições que, após consumir toda a entrada, esvazia completamente a pilha. Formalmente:

$$N(AP) = \{ w \in \Sigma \mid (q_0, w, Z_0) \vdash (q, \epsilon, \epsilon) \text{ para algum } q \in Q \}^{**}$$

Cada configuração (q, w, γ) representa: estado atual, entrada restante e conteúdo da pilha. A aceitação ocorre quando a entrada restante é ϵ e a pilha também é ϵ — ou seja, até o marcador Z_0 foi removido.

Observações importantes:

- O estado q ao final pode ser qualquer estado; F pode ser $\emptyset$.
- A aceitação por pilha vazia é equivalente em poder à aceitação por estado final — ambas reconhecem exatamente as linguagens livres de contexto.
- Como δ é não-determinística, basta que exista ao menos um caminho de computação que esvazie a pilha.

CASOS DE TESTE

Linguagem: $L_2 = \{ a^n b^n \mid n \geq 1 \}$

Essa linguagem é formada por cadeias com n letras 'a' seguidas de exatamente n letras 'b', com $n \geq 1$.

Cadeia: ab

$n = 1$, portanto a cadeia deve ter 1 letra 'a' e 1 letra 'b'.

- Passo 1: Lê o símbolo 'a' $\rightarrow$ conta 1 'a'
- Passo 2: Lê o símbolo 'b' $\rightarrow$ conta 1 'b'
- Passo 3: Fim da cadeia. Total: 1 'a' e 1 'b'
- Verificação: $n = 1 \rightarrow a^1b^1 = ab \checkmark$

Conclusão: "ab" $\in L_2$, pois satisfaz a condição com $n = 1$.

Cadeia: aabb

$n = 2$, portanto a cadeia deve ter 2 letras 'a' e 2 letras 'b'.

- Passo 1: Lê o símbolo 'a' $\rightarrow$ conta 1 'a'
- Passo 2: Lê o símbolo 'a' $\rightarrow$ conta 2 'a'
- Passo 3: Lê o símbolo 'b' $\rightarrow$ conta 1 'b'
- Passo 4: Lê o símbolo 'b' $\rightarrow$ conta 2 'b'
- Passo 5: Fim da cadeia. Total: 2 'a' e 2 'b'
- Verificação: $n = 2 \rightarrow a^2b^2 = aabb \checkmark$

Conclusão: "aabb" $\in L_2$, pois satisfaz a condição com $n = 2$.

2.3 Máquina de Turing (MT)

A Máquina de Turing (MT) é definida formalmente pela 7-tupla $M = (Q, \Sigma, \Gamma, \delta, q_0, B, F)$, onde Q é o conjunto finito de estados, Σ é o alfabeto de entrada, Γ é o alfabeto da fita, $\delta: Q \times \Gamma \rightarrow Q \times \Gamma \times \{E, D, P\}$ é a função de transição parcial (com as direções Esquerda, Direita ou Parada), q_0 é o estado inicial, $B \in \Gamma$ é o símbolo branco e $F \subseteq Q$ é o conjunto de estados de aceitação. Diferente dos autômatos anteriores, a fita é bidirecional e ilimitada, e a ausência de uma transição definida em δ para o cenário atual causa a parada imediata e rejeição da cadeia.

Para reconhecer a linguagem sensível ao contexto $L_4 = \{ a^n b^n c^n \mid n \geq 1 \}$, adotou-se a estratégia de marcação em passadas sucessivas. A cada ciclo, a MT varre a fita da esquerda para a direita substituindo o primeiro 'a' por 'X', o primeiro 'b'

por 'Y' e o primeiro 'c' por 'Z'. Após marcar o 'Z', o cabeçote retorna para a esquerda até encontrar a fronteira do último 'X' marcado e recomeça o ciclo. Ao final, quando não restarem mais 'a's para leitura, a máquina realiza uma varredura residual para garantir que não sobraram 'b's ou 'c's desbalanceados, aceitando a cadeia ao encontrar o primeiro símbolo branco.

Cálculo manual para a cadeia "abc" (esperado: ACEITA):

Passo	Estado Atual	Conteúdo da Fita (Cabeça destacada)	Ação de Transição
1	q0	[a] b c	Lê 'a', grava 'X', move p/ Direita
2	q1	X [b] c	Lê 'b', grava 'Y', move p/ Direita
3	q2	X Y [c]	Lê 'c', grava 'Z', move p/ Esquerda
4	q3	X [Y] Z	Lê 'Y', retorna p/ Esquerda
5	q3	[X] Y Z	Lê 'X', altera p/ q0, move p/ Direita
6	q0	X [Y] Z	Lê 'Y', altera p/ q4, move p/ Direita
7	q4	X Y [Z]	Lê 'Z', altera p/ q5, move p/ Direita
8	q5	X Y Z []	Lê Branco, finaliza em qac → ACEITA

Cálculo manual parcial para a cadeia "aabbcc" (esperado: ACEITA):

Passo	Estado Atual	Conteúdo da Fita (Cabeça destacada)	Observação
1 a 3	q0 → q2	[a] a b b c c → X a Y b [c] c	Fim do 1º ciclo de marcação.

Passo	Estado Atual	Conteúdo da Fita (Cabeça destacada)	Observação
4 a 6	$q_3 \rightarrow q_0$	X a [Y] b Z c $\rightarrow$ X [a] Y b Z c	Retorno da cabeça ao início do 2º ciclo.
7 a 11	$q_0 \rightarrow q_2$	X [a] Y b Z c $\rightarrow$ X X Y Y Z [c]	Fim do 2º ciclo de marcação.
12 a 15	$q_3 \rightarrow q_5$	X X Y Y [Z] c $\rightarrow$ X X Y Y Z Z []	Varredura final de verificação $\rightarrow$ ACEITA.

3. Descrição da Implementação

3.1 Parte 1 - AFD

O simulador do AFD foi implementado em C# (.NET 8). Cada campo da 5-tupla foi declarado explicitamente com a estrutura de dados que melhor representa sua semântica matemática. Q e F foram implementados como `HashSet<string>`, pois na teoria são conjuntos — estruturas sem duplicatas e sem ordem definida. O `HashSet` garante exatamente esse comportamento: inserir um estado repetido não tem efeito, e a verificação de pertinência (`Contains`) é $O(1)$. O alfabeto Σ foi declarado como `HashSet<char>` pelo mesmo motivo.

A função de transição δ foi implementada como `Dictionary<(string estado, char simbolo), string>`, onde a chave é uma `ValueTuple` com o estado atual e o símbolo lido, e o valor é o próximo estado. Essa estrutura reflete diretamente a definição matemática de $\delta : Q \times \Sigma \rightarrow Q$ — cada par (estado, símbolo) tem exatamente um destino.

O método `Aceitar` percorre a cadeia símbolo a símbolo, mantendo o estado atual e registrando o rastro de estados visitados em uma `List<string>`. A cada passo, verifica se o símbolo pertence a Σ antes de aplicar δ — tratando o caso de entrada inválida sem lançar exceção. Ao final, retorna se o estado atual pertence a F.

O método `ExibirDiagrama` imprime no console a tabela de transições δ

formatada, marcando o estado inicial com '>' e o estado de aceitação com '*'. O método ProcessarArquivo lê o arquivo entradas.txt linha a linha e chama Aceitar para cada cadeia, exibindo a cadeia, o rastro e o resultado.

O desafio obrigatório foi implementado no método ProcessaJson, que carrega qualquer AFD a partir de um arquivo afd.json usando System.Text.Json. O método reconstrói a 5-tupla completa a partir do JSON e simula as mesmas cadeias do entradas.txt com o AFD carregado dinamicamente, sem alterar o código-fonte.

3.2 Parte 2 — Autômato de Pilha

A implementação do simulador de AP em C# utiliza a classe genérica Stack<char> para representar a memória LIFO. Diferente do AFD, o motor do AP precisa gerenciar **e-movimentos** (transições que não consomem símbolos da entrada), o que foi implementado através de uma verificação prioritária: antes de ler o próximo caractere da cadeia, o simulador checa se existe uma transição válida para o símbolo vazio (ε).

As configurações instantâneas são exibidas no console a cada passo, mostrando o estado atual, a subcadeia restante e o conteúdo da pilha (ex: [A, A, Z]). Isso permite validar visualmente o crescimento e decréscimo da memória durante o processamento.

Para o desafio do **AP de Palíndromos (L3)**, a implementação lida com o não-determinismo inerente à identificação do meio da cadeia (em palíndromos de comprimento par ou ímpar). A solução adotada utiliza um "marcador de meio" ou uma lógica de tentativa de transição para o estado de desempilhamento, simulando o comportamento não-determinístico através de verificações de estado para garantir que cadeias como "aba" ou "abba" sejam reconhecidas corretamente ao espelhar os símbolos da pilha com os da entrada.

3.3 Parte 3 — Máquina de Turing

A arquitetura do simulador da Máquina de Turing foi projetada para isolar o motor de execução (MaquinaDeTuring.cs) das regras específicas de cada linguagem. Para representar a fita infinita bidirecional sem os custos de realocação

de vetores tradicionais, optou-se pela estrutura `Dictionary<int, char>`. Nesta abordagem, a chave representa o índice da fita (permitindo índices negativos para movimentos contínuos à esquerda) e o valor armazena o símbolo. Células que nunca foram visitadas ou gravadas não consomem memória; um método de leitura seguro avalia chaves inexistentes retornando dinamicamente o símbolo branco ('_').

A função de transição δ foi implementada usando a nova estrutura do .NET 8, `FrozenDictionary<(string, char), (string, char, char)>`. Esse formato imutável oferece buscas em complexidade $O(1)$. A matemática das funções parciais de Turing foi traduzida nativamente: caso o método falhe em encontrar a tupla correspondente ao estado e símbolo atuais, a máquina é imediatamente interrompida e a cadeia rejeitada, descartando a necessidade de programar múltiplos "estados de erro" para lixo sintático.

O laço de simulação opera em um ciclo contínuo de leitura e escrita. Para mitigar problemas de indecidibilidade sistêmica (o Paradoxo do Problema da Parada), implementou-se um contador âncora com `MaxPassos = 10000`. Se uma máquina entrar em loop infinito, a execução é abortada retornando `LimiteExcedido`. A interface no console conta com um atraso (delay) de 150 milissegundos a cada transição para garantir a correta exibição visual passo a passo da fita.

Por fim, o modelo também foi instanciado como uma MT transdutora para computar a função matemática $f(n) = n + 1$ operando no sistema unário. Diferente da base binária, em que a soma exige lógicas complexas de transporte da direita para a esquerda, a MT unária requer apenas 2 estados: ela varre iterativamente a fita da esquerda para a direita no estado q_0 ignorando os símbolos '1' e, ao encontrar o primeiro símbolo branco, transita para q_{ac} gravando o caractere '1' adicional, finalizando o algoritmo.

4. Resultados e Testes

4.1 Parte 1 - AFD

A tabela a seguir apresenta os resultados obtidos pela execução do simulador para os 7 casos de teste obrigatórios. Todos produziram o resultado esperado.

Tabela 1 — Função de transição δ do AFD para L_1

Estado	'a'	'b'
$\rightarrow q_0$ (inicial)	q_1	q_0
q_1	q_1	q_2
$*q_2$ (aceitação)	q_1	q_0

Tabela 2 — Resultados dos casos de teste para L_1

Cadeia	Esperado	Rastro de Estados
ab	ACEITA	$q_0 \rightarrow q_1 \rightarrow q_2$
aab	ACEITA	$q_0 \rightarrow q_1 \rightarrow q_1 \rightarrow q_2$
bab	ACEITA	$q_0 \rightarrow q_0 \rightarrow q_1 \rightarrow q_2$
ababab	ACEITA	$q_0 \rightarrow q_1 \rightarrow q_2 \rightarrow q_1 \rightarrow q_2 \rightarrow q_1 \rightarrow q_2$
ba	REJEITA	$q_0 \rightarrow q_0 \rightarrow q_1$
ϵ	REJEITA	q_0 (sem transições)
b	REJEITA	$q_0 \rightarrow q_0$

4.2 Parte 2 — Autômato de Pilha

Cadeia	Esperado	Resultado	Justificativa
"ab"	ACEITA	ACEITA	n = 1
"aabb"	ACEITA	ACEITA	n = 2
"aaabbb"	ACEITA	ACEITA	n = 3
"aab"	REJEITA	REJEITA	Contagem desigual (2a, 1b)
"abb"	REJEITA	REJEITA	Contagem desigual (1a, 2b)
"ba"	REJEITA	REJEITA	Ordem invertida
e	REJEITA	REJEITA	Exige $n \geq 1$
"abab"	REJEITA	REJEITA	Formato intercalado inválido

Linguagem L3:

Cadeia	Esperado	Resultado
"a"	ACEITA	ACEITA

"aba"	ACEITA	ACEITA
"abba"	ACEITA	ACEITA
"ab"	REJEITA	REJEITA
"aab"	REJEITA	REJEITA

4.3 Parte 3 — Máquina de Turing

As tabelas a seguir apresentam os resultados da simulação para a Máquina de Turing reconhecedora de L_4 e para a Máquina de Turing transdutora da função $f(n) = n + 1$. O sistema identificou corretamente cadeias incompletas, ordens inválidas e excedentes sintáticos, provando a eficácia da interrupção da função parcial δ .

Tabela 3 — Resultados dos testes da MT Reconhecedora para L_4

Cadeia de Entrada	Análise Sintática	Resultado Esperado
"abc"	$n = 1 \in L_4$	ACEITA
"aabbcc"	$n = 2 \in L_4$	ACEITA
"aaabbbccc"	$n = 3 \in L_4$	ACEITA
"aabbcc"	Desbalanceamento (falta 'c')	REJEITA
"abcabc"	Ordem de símbolos inválida	REJEITA
"aabbccc"	Excedente de caracteres finais	REJEITA

Cadeia de Entrada	Análise Sintática	Resultado Esperado
ε (Vazia)	Exigência teórica de $n \geq 1$	REJEITA

Tabela 4 — Resultados dos testes da MT Transdutora para $f(n) = n + 1$ (Unário)

Fita Inicial Fornecida	Valor de 'n'	Saída Esperada na Fita
ε	0	"1"
"1"	1	"11"
"111"	3	"1111"
"11111"	5	"111111"

5. Análise Comparativa

Os três modelos estudados neste trabalho formam uma hierarquia de poder computacional conhecida como Hierarquia de Chomsky, na qual cada nível estritamente contém o anterior. A tabela a seguir resume as principais diferenças entre as máquinas:

Tabela 5 — Comparativo entre AFD, Autômato de Pilha e Máquina de Turing

Característica	AFD	Autômato de Pilha	Máquina de Turing
Memória	Nenhuma (estados)	Pilha (LIFO)	Fita ilimitada
Classe de linguagens	Regulares	Livres de contexto	Recursivamente enumeráveis
Exemplo reconhecido	$(a b)^*ab$	$a^n b^n$	$a^n b^n c^n$
Linguagem não reconhecida	$a^n b^n$	$a^n b^n c^n$	Problema da parada
Determinismo	Sempre	Pode ser não-det.	Determinístico

Característica	AFD	Autômato de Pilha	Máquina de Turing
	determinístico		
Poder computacional	Menor	Intermediário	Máximo (Tese de Church-Turing)

5.1 O que o AFD não consegue fazer

O AFD é o modelo mais restrito dos três. Sua limitação fundamental é a ausência de memória auxiliar: o autômato só pode "lembrar" informações codificadas em seus próprios estados, que são em número finito. Por esse motivo, o AFD não consegue reconhecer linguagens que exigem contagem ilimitada ou verificação de balanceamento.

A linguagem $L_2 = \{ a^n b^n \mid n \geq 1 \}$, reconhecida pelo Autômato de Pilha neste trabalho, é um exemplo clássico de linguagem não regular. O Lema do Bombeamento para linguagens regulares demonstra formalmente que nenhum AFD pode reconhecê-la: para qualquer AFD hipotético para L_2 , seria possível "bombear" o segmento inicial de a's, gerando cadeias como $a^k b^n$ com $k \neq n$ que deveriam ser aceitas mas não são (Sipser, 2013).

5.2 O que o Autômato de Pilha não consegue fazer

O Autômato de Pilha supera o AFD ao adicionar uma memória de pilha, o que permite reconhecer linguagens livres de contexto como $a^n b^n$. No entanto, a pilha é uma estrutura de acesso restrito (apenas o topo é acessível), o que impede o reconhecimento de linguagens que exigem múltiplos contadores independentes e simultâneos.

A linguagem $L_4 = \{ a^n b^n c^n \mid n \geq 1 \}$, reconhecida pela Máquina de Turing neste trabalho, não pode ser reconhecida por nenhum Autômato de Pilha. Isso ocorre porque reconhecer L_4 exige verificar simultaneamente que os grupos de a's, b's e c's têm o mesmo tamanho — o que requer dois contadores independentes, algo que uma única pilha não suporta (Hopcroft, Motwani e Ullman, 2003).

5.3 O poder da Máquina de Turing

A Máquina de Turing é o modelo computacional mais poderoso entre os três. Sua fita ilimitada funciona como uma memória de acesso aleatório, e o cabeçote pode se mover livremente em ambas as direções, permitindo múltiplas passagens sobre a entrada. Isso a torna capaz de reconhecer todas as linguagens recursivamente enumeráveis e de computar qualquer função computável.

A Tese de Church-Turing estabelece que qualquer processo computacional que possa ser descrito por um algoritmo pode ser simulado por uma Máquina de Turing. Isso não significa que a MT resolve todos os problemas: o Problema da Parada — decidir se uma MT arbitrária para em uma dada entrada — é indecidível, demonstrando que mesmo o modelo mais poderoso possui limites fundamentais (Sipser, 2013).

Neste trabalho, a MT implementada para L_4 ilustra concretamente esse poder superior: ela realiza múltiplas varreduras da fita, marcando progressivamente os símbolos a, b e c de forma sincronizada, algo impossível para qualquer Autômato de Pilha.

6. Conclusão

Este trabalho permitiu consolidar a compreensão das três principais máquinas abstratas da Teoria da Computação por meio de uma abordagem prática: a implementação de simuladores funcionais em C#. Ao traduzir as definições matemáticas formais — 5-tuplas e 7-tuplas — diretamente em estruturas de dados e algoritmos, tornou-se concreta a relação entre teoria e código.

A implementação do AFD evidenciou como um modelo de memória nula consegue reconhecer padrões regulares de forma eficiente, mas também revelou seus limites ao perceber que qualquer linguagem que exija contagem está além de seu alcance. O Autômato de Pilha demonstrou como a adição de uma estrutura de memória simples — a pilha — expande substancialmente o poder de reconhecimento, ao custo de uma lógica de controle mais complexa com λ -movimentos e aceitação por pilha vazia.

A Máquina de Turing, por sua vez, mostrou-se o modelo mais desafiador de implementar, mas também o mais poderoso: a fita ilimitada e a capacidade de reescrever símbolos permitem estratégias de reconhecimento impossíveis nos modelos anteriores, como a marcação sincronizada de múltiplos grupos de símbolos em L_4 .

Do ponto de vista do desenvolvimento, os commits incrementais no GitHub e a exibição passo a passo das configurações instantâneas foram fundamentais para depuração e validação dos simuladores. A hierarquia de poder computacional — $AFD \subset AP \subset MT$ — deixou de ser apenas um conceito teórico para se tornar algo verificável por meio dos próprios casos de teste do trabalho.

Por fim, este trabalho reforça que os limites dos modelos computacionais não são apenas acadêmicos: eles têm implicações diretas no design de linguagens de programação, compiladores e algoritmos. Compreender o que cada modelo pode e não pode fazer é uma competência fundamental para qualquer profissional de computação.

7. Referências

SIPSER, Michael. *Introduction to the Theory of Computation*. 3. ed. Boston: Cengage Learning, 2013.

HOPCROFT, John E.; MOTWANI, Rajeev; ULLMAN, Jeffrey D. *Introdução à Teoria de Autômatos, Linguagens e Computação*. 2. ed. Rio de Janeiro: Campus/Elsevier, 2003.

MENEZES, Paulo Blauth. *Linguagens Formais e Autômatos*. 6. ed. Porto Alegre: Bookman, 2010.

LEWIS, Harry R.; PAPADIMITRIOU, Christos H. *Elements of the Theory of Computation*. 2. ed. Upper Saddle River: Prentice Hall, 1998.